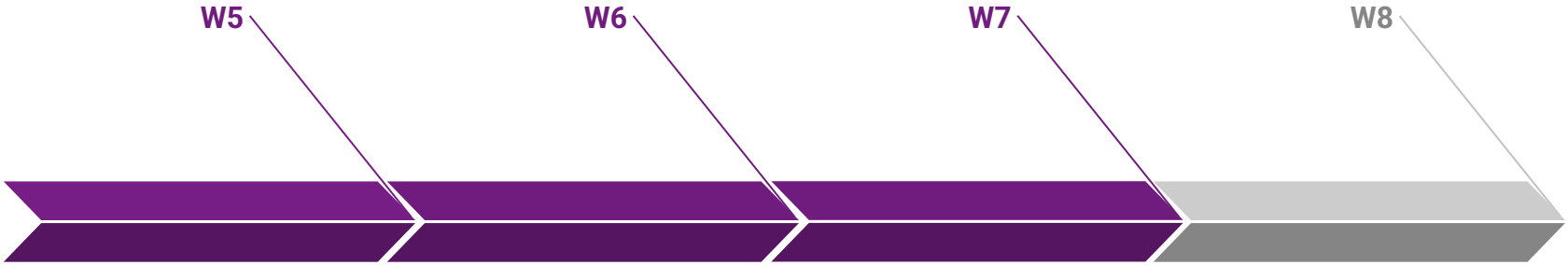
Software Engineering

Typescript Basics

Module 3: Javascript and Typescript



Beginner Javascript

After mastering HTML and CSS, students begin programming with Beginner JavaScript . In this module, they learn the basics of how to make web pages interactive and dynamic using JavaScript, the main programming language for front-end development.

Advanced Javascript

Once comfortable with the fundamentals, students continue to Advanced JavaScript . This module covers more complex features, tools, and best practices, enabling them to build more sophisticated and efficient web applications.

Typescript

Next, TypeScript introduces students to a modern version of JavaScript that adds type safety and other powerful features. Learning TypeScript helps students write cleaner and more reliable code, which is important in professional development.

Front-end Infrastructure and AI Integration

Shows how modern websites are built using new technologies and tools, and how AI can be used with them. This module ties together everything learned earlier and prepares students for real-world web development.

What we will learn...

- Introduction to Typescript
- Difference between Typescript and Javascript
- Understanding Typescript Nature and Syntax

✋ Any first questions?

Introduction to Typescript

- TypeScript is a programming language that extends JavaScript by adding static types to the language.
- TypeScript is a superset of JavaScript, which means that any valid JavaScript code is also valid TypeScript code.
- However, TypeScript introduces additional features, including static typing, interfaces, and advanced tooling support, to enhance the development experience and improve code quality.

Typescript, for what ?

Typescript, for what ?

https://i.pinimg.com/originals/a4/42/6f/a4426f32434eada5c73f1b24c2c84340.jpg

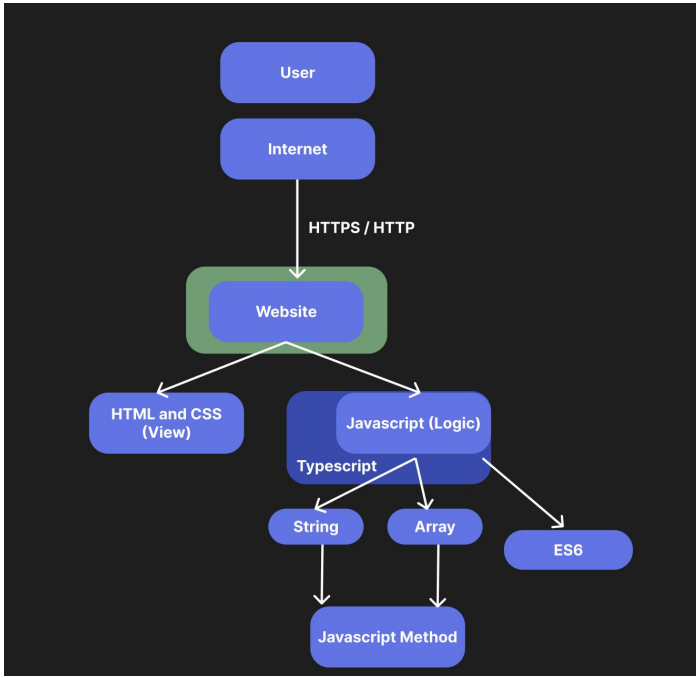
Why ?

https://media.istockphoto.com/id/1282657902/vector/business-decision-making-career-path-work-direction-or-leadership-to-choose-the-right-way-to.jpg?s=612x612&w=0&k=20&c=P2agdcoukbJrpa-Ab2FAYcqSSZQlEAYLVacDf1c2J0U=

Why ?

shutterstock.com · 1829532161

Typescript


User
Internet
HTTPS / HTTP
Website
HTML and CSS
(View)
Javascript (Logic)
Typescript
String
Array
ES6
Javascript Method

Why adding types important ?

- TypeScript is a programming language that extends JavaScript by adding static types to the language.
- TypeScript is a superset of JavaScript, which means that any valid JavaScript code is also valid TypeScript code.
- However, TypeScript introduces additional features, including static typing, interfaces, and advanced tooling support, to enhance the development experience and improve code quality.

TypeScript vs JavaScript

TypeScript vs JavaScript

TypeScript and JavaScript are both programming languages, but they have some key differences in terms of features, use cases, and how they are used in the development process.

1. Type System:
JavaScript: It is a dynamically typed language, meaning that variable types are determined at runtime. This flexibility allows for more concise and flexible code but can lead to runtime errors due to type mismatches.

TypeScript: It is a statically typed language, meaning that variable types are checked at compile-time. TypeScript introduces static typing, allowing developers to define the types of variables, function parameters, and return types. This helps catch type-related errors early in the development process.

2. Code Structure:
JavaScript: It is a scripting language that follows the ECMAScript standard. It has a more relaxed syntax and is generally less strict in terms of code organization.

TypeScript: It is a superset of JavaScript, which means any valid JavaScript code is also valid TypeScript code. TypeScript introduces additional features like interfaces, enums, and decorators, encouraging a more structured and object-oriented approach to coding.

TypeScript vs JavaScript

3. Tooling:
JavaScript: Development tools for JavaScript are generally less sophisticated compared to TypeScript. While modern IDEs provide features like autocompletion and debugging, JavaScript lacks the comprehensive static analysis that TypeScript offers.

TypeScript: TypeScript comes with enhanced tooling support. Modern IDEs, such as Visual Studio Code and others, provide features like intelligent code completion, type checking, and advanced refactoring options, making the development experience more robust.

4. Compatibility:
JavaScript: It is supported in all browsers and can be executed on any JavaScript runtime.

TypeScript: TypeScript code needs to be transpiled to JavaScript before it can be executed in a browser. TypeScript code can coexist with JavaScript code, allowing for a gradual migration of existing projects.

TypeScript vs JavaScript

5. Use Cases:
JavaScript: Widely used for web development, both on the client side (browser) and server side (Node.js). It is also used in various other contexts, including mobile app development (using frameworks like React Native), game development, and more.

TypeScript: Commonly used in large-scale applications where static typing and code organization are crucial. It is widely used in modern web development frameworks and libraries like Angular, NestJS, and others.

Conclusion:
The choice between TypeScript and JavaScript often depends on the project requirements, team preferences, and the nature of the application being developed. JavaScript is well-suited for smaller projects and scenarios where flexibility is crucial, while TypeScript excels in large-scale applications where static typing and advanced tooling can contribute to better code quality and maintainability. Some projects may even use both languages in different parts of the codebase to leverage the strengths of each.

Why adding types important ?

Why use TypeScript?

Using TypeScript offers several advantages for developers and projects, particularly in larger and more complex applications. Here are some reasons why developers choose to use TypeScript:

Static Typing: TypeScript introduces static typing, catching many common errors at compile-time, leading to improved code reliability.

Code Maintainability: TypeScript encourages a structured coding style with features like interfaces, making code more organized and maintainable.

Tooling and IDE Support: Advanced tooling support in TypeScript, such as autocompletion and type checking, enhances the development experience in modern IDEs.

Early Error Detection: Static typing allows the TypeScript compiler to catch errors early in the development process, reducing bugs and improving overall code quality.

Compatibility and Collaboration: TypeScript is compatible with existing JavaScript code, making it easy to adopt gradually. It promotes better collaboration in teams working on larger projects and provides a stronger ecosystem with TypeScript support in popular frameworks and libraries.

Installation

Installation TypeScript

To install TypeScript, you can use the Node Package Manager (npm) if you have Node.js installed. Here are the steps to install TypeScript globally on your machine:

1. Install Node.js and npm:

Before installing TypeScript, ensure you have Node.js and npm installed on your machine. You can download and install them from the official Node.js website: Node.js Downloads

2. Open a Terminal or Command Prompt:

Open a terminal or command prompt on your computer.

3. Install TypeScript using npm:

Run the following command to install TypeScript globally:

```
npm install -g typescript
```

The -g flag installs TypeScript globally, making it available as a command-line tool.

Installation TypeScript

4. Verify the Installation:

After the installation is complete, you can verify the TypeScript installation by checking the installed version:

```
tsc --version
```

This command should print the installed TypeScript version.

Now you have TypeScript installed on your machine, and you can use the tsc (TypeScript Compiler) command to compile TypeScript files (*.ts) into JavaScript files (*.js). You can also use an integrated development environment (IDE) like Visual Studio Code, which provides excellent support for TypeScript out of the box.

Compiling TypeScript

1. Create a TypeScript file: Create a .ts file with your TypeScript code.
For example, let's say you have a file named script.ts containing TypeScript code.

2. Compile TypeScript to JavaScript:
Open a terminal or command prompt, navigate to the directory containing your TypeScript file, and run the TypeScript compiler (tsc) with the name of your TypeScript file as the argument:

```
`tsc script.ts`
```

This will compile your TypeScript code into JavaScript. The compiler will generate a corresponding .js file with the same name as your TypeScript file (in this case, script.js).

3. After you compile it’s automatically compiles it into file with same name but in JS extension.
script.js file contain compiled file from script.ts

Typescript Nature and Syntax

Types in TypeScript

TypeScript inherits the built-in types from JavaScript. TypeScript types is categorized into:

- Primitive types

`string`	represents text data
`number`	represents numeric values
`boolean`	has true and false values
`null`	has one value: null
`undefined`	has one value: `undefined` . It is a default value of an uninitialized variable
`symbol`	represents a unique constant value

- Object types
  Object types are functions, arrays, classes, etc. Later, you'll learn how to create custom object types.

Types Annotations

TypeScript employs type annotations to precisely define the types of identifiers, such as variables, functions, and objects, among others.

The type annotation syntax in TypeScript involves placing a colon (:) followed by the desired type after the identifier. This syntax allows any valid type to be specified as the type annotation.

```
let variableName: type;
let variableName: type = value;
const constantName: type = value;
```

```
let counter: number;

counter = 1; // Okay

counter = "RevoU" // compile error
```

Types Annotations

```
Type 'string' is not assignable to type 'number'. ts(2322)
let counter: number
View Problem (⌥F8)    No quick fixes available
counter = "RevoU"
```

You can both use a type annotation for a variable and initialize it in a single statement like this:

```
let name: string = 'Yuhuu';
let age: number = 25;
let active: boolean = true;
```

Types Inference

Type inference describes where and how TypeScript infers types when you don’t explicitly annotate them.

When you declare a variable, you can use a type annotation to explicitly specify a type for it. For example:

```
let number: number;
```

Types Inference

However, if you initialize the counter variable to a number, TypeScript will infer the type the counter to be number. For example:

```
let number = 0;
```

It is equivalent to the following statement:

```
let number: number = 0;
```

Type inference vs. Type annotations

Type inference	Type annotations
TypeScript guesses the type	You explicitly tell TypeScript the type

Number

You still can use number type for Decimal, Binary, Hexadecimal and Octal number

Except Big Int you need declare as “bigint”

```
let counter: number = 0;
let binary: number = 0B010;
let hexadecimal: number = 0XA0XA;
let octal: number = 0o10;

let big: bigint = 9007199254740991n;
```

String

Similar to JavaScript, TypeScript utilizes either double quotes (") or single quotes (') to enclose string literals.

TypeScript also supports template strings that use the backtick (`) to surround characters and string interpolations.

```
let firstName: string = 'Peter';
let title: string = "Full Snack Developer";
let description = `I love coding and
snack`;

let profile: string = `I'm ${firstName}.
I'm a ${title}`;
```

Boolean

The boolean type in TypeScript represents a data type that can hold one of two values: true or false. It is classified as one of the primitive types available in TypeScript. For instance:

```
let isLoading: boolean;
isLoading = true;
// after a while
// ..
isLoading = false;
```

Objects

The TypeScript object type represents all values that are not in primitive types.

```
let employee: {
  name: string;
  age: number
};

employee = {
  name: "Peter Parker",
  age: 17
}
```

Objects

Or you can combine both syntaxes in the same statement like this:

```
let employee: {
  name: string;
  age: number
} = {
  name: "Peter Parker",
  age: 17
}
```

Arrays

In TypeScript, an array is a sequential collection of data elements. To define an array that stores values of a particular type, you can use the following syntax:

```
let arrayName: type[];

let names: string[] = ['Mary', 'Jane', 'Peter',
'Parker', 'May', "Eddy", "Brooke"];
```

You still can use Array Method like push()

```
names.push('Harry Osborn')
```

Arrays

Once you define an array of a specific type, TypeScript will prevent you from adding incompatible values to the array.

```
names.push(0)
// Argument of type 'number' is not assignable
to parameter of type 'string'.
```

We want to save various data with string & number so we can do this

```
let names : (string | number)[];
names = ['JavaScript', 100, 'RevoU', 40];
```

Tuples

- A tuple works like an array with some additional considerations.
- The number of elements in the tuple is fixed.
- The types of elements are known, and need not be the same.

```
let skill: [string, number];
skill = ['Learn', 99];
```

- The order of values in a tuple is important.
- If you change the order of values of the skill tuple, you'll get an error:

```
let skill: [string, number];
skill = [99, 'Learn'];
```

Any

Occasionally, there might be situations where you need to assign a value to a variable, but you're unsure of its type during the development stage. This unknown value could originate from an external API or user input.

To bypass type checking and permit the value to pass through the compile-time checks, you can utilize the any type. By using the any type, you can assign a value of any type to a variable, allowing for greater flexibility:

```
// json may come from a third-party API
const json = `{"latitude": 6.1754° S,
"longitude":106.8272° E}`;

// parse JSON to find location
const currentLocation = JSON.parse(json)
console.log(currentLocation);
```

Any

Using the any type in TypeScript should be minimized due to the following reasons:

Type Safety: any bypasses type checking, compromising the benefits of TypeScript's strong type system and increasing the likelihood of runtime errors.

Compiler Benefits: any hinders the usefulness of TypeScript's compiler features, such as autocompletion, error detection, and refactoring support.

Debugging Challenges: Tracking down issues related to type mismatches becomes more difficult, as the lack of type information complicates the debugging process.

Union

Sometimes, you will run into a program that expects a parameter or result that is either a number or a string. For example:

```
let result: number | string;
result = 100; // OK
result = 'Haloo'; // also OK
result = false; // a boolean value, not OK
```

Function

In TypeScript, functions serve as fundamental components for creating code that is easy to read, maintain, and reuse.

Similar to JavaScript, the function keyword is utilized in TypeScript to declare a function.

```
function functionName(parameter: type, parameter:type,...): returnType {
   // do something
}

// example

function add(a: number, b: number): number {
    return a + b;
}
```

In this given example, the add() function takes two parameters of type number.

Function

When invoking the add() function, the TypeScript compiler verifies that each argument provided to the function is indeed a number.

In the provided add() function example, it only allows numbers to be passed as arguments and not values of other types.

The subsequent code will produce an error since it attempts to pass two strings instead of two numbers into the add() function:

```
let sum = add('10', '20');
```

The presence of ": number" after the parentheses denotes the return type specification. In this particular case, the add() function is expected to return a value of the number type.

When a function has a designated return type, the TypeScript compiler examines each return statement to ensure that the returned value aligns with the specified type.

Void Function

If a function does not return a value, you can use the void type as the return type. The void keyword indicates that the function doesn’t return any value. For example:

```
function lowerCase(message: string): void {
    console.log(message.toLowerCase());
}
```

The usage of "void" within the function prevents any value from being returned and restricts the calling code from assigning the function's result to a variable.

Function Inference

When the return type is not explicitly annotated, TypeScript will attempt to deduce a suitable type through inference. For instance:

```
function add(a: number, b: number) {
    return a + b;
}
```

In this example, TypeScript tries to figure out that the return type of the add() function is a number, which is what we expect.

However, if a function has different branches that return different types, TypeScript might infer a union type or the any type.

Function Optional Parameters

In JavaScript, it is possible to call a function without providing any arguments, even if the function has defined parameters. This means that JavaScript inherently supports optional parameters.

However, TypeScript operates differently. The TypeScript compiler examines every function call and generates an error in two scenarios:

- The number of arguments provided differs from the number of parameters defined in the function.
- The types of the arguments are incompatible with the types of the function parameters.

Function Optional Parameters

The compiler carefully checks the arguments that are passed, so you must annotate optional parameters to prevent the compiler from throwing an error when arguments are omitted.

To make a parameter optional in a function, you can add a "?" after its name. For instance:

```
function add(a: number, b: number, c?: number): number {
    if (typeof c !== null) {
        return a + b + c;
    }
    return a + b;
}
```

The parameter "c" is optional because it has a "?" after its name.

Function Default Parameters

JavaScript supported default parameters, but how we use it with TypeScript?

```
function finalPricetJS(price, discount = 0) {
    return price * (1 - discount);
}


function finalPriceTS(price: number, discount: number = 0): number {
    return price * (1 - discount);
}

console.log(finalPriceJS(100)); // 95
console.log(finalPriceTS(100)); // 95
```

Function Rest Parameters

In TypeScript, a rest parameter lets a function accept any number of arguments of a specific type. Here are the rules for using rest parameters:

- A function can have only one rest parameter.
- The rest parameter should be the last parameter in the function.
- The rest parameter is declared by putting three dots (...) before its name and using an array type for its type.

In simple terms, a rest parameter allows a function to handle multiple arguments of the same type, and it is indicated by using three dots (...) before the parameter name and specifying an array type for the parameter's type.

```
function fn(...rest: type[]) {
   // Do Something
}
```

Function Rest Parameters

```
function totalNum(...numbers: number[]): number {
    let total = 0;
    numbers.forEach((num) => total += num);
    return total;
}

console.log(totalNum()); // 0
console.log(totalNum(10, 20)); // 30
console.log(totalNum(10, 20, 30)); // 60
```

QnA & Discussion

Today’s Recap

External Resources

- 📖 **Introduction to TypeScript:**
  - TypeScript Documentation
  - TypeScript Playground
  - Academind TypeScript Tutorial
- 📖 **Understanding TypeScript's Nature and Syntax:**
  - TypeScript Handbook
  - TypeScript Deep Dive
- 📖 **Difference Between TypeScript and JavaScript:**
  - Stack Overflow - TypeScript vs. JavaScript
  - TypeScript vs. JavaScript

Thank you!

There is a certain amount of flexibility where it is the strongest security.
There is a certain amount of strictness that can be the worst security.

Share your learning journey with us!
Let's inspire each other with your stories

@revou_id RevoU revoudotco